

SALUS SECURITY

JAN 2026



CODE SECURITY ASSESSMENT

ASPECTA AI

Overview

Project Summary

- Name: Aspecta - AI
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
 - <https://github.com/Lorenzo-Protocol/earn-contract>
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Aspecta - AI
Version	v2
Type	Solidity
Dates	Jan 20 2026
Logs	Jan 19 2026, Jan 20 2026

Vulnerability Summary

Total High-Severity issues	2
Total Medium-Severity issues	5
Total Low-Severity issues	1
Total informational issues	1
Total	9

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Settlement for long positions may be blocked	6
2. Settlement failure via manipulate options	7
3. Temporary Dos via donating	8
4. Potential Asset Loss for AA Wallets via Cross-chain Settlement	9
5. Unauthorized Trade Execution via tx.origin Phishing	10
6. Pool Insolvency due to Bonding Curve Price Hike via setPiecewisess	11
7. Centralization risk	12
8. Missing whenNotPaused Modifier in openShort and closeShort functions	13
2.3 Informational Findings	14
9. Redundant code	14
Appendix	15
Appendix 1 - Files in Scope	15

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Settlement for long positions may be blocked	High	Business Logic	Resolved
2	Settlement failure via manipulate options	High	Business Logic	Resolved
3	Temporary Dos via donating	Medium	Business Logic	Acknowledged
4	Potential Asset Loss for AA Wallets via Cross-chain Settlement	Medium	Business Logic	Resolved
5	Unauthorized Trade Execution via tx.origin Phishing	Medium	Business Logic	Acknowledged
6	Pool Insolvency due to Bonding Curve Price Hike via setPiecwises	Medium	Business Logic	Acknowledged
7	Centralization risk	Medium	Centralization	Acknowledged
8	Missing whenNotPaused Modifier in openShort and closeShort functions	Low	Business Logic	Resolved
9	Redundant code	Informational	Configuration	Resolved

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Settlement for long positions may be blocked	
Severity: High	Category: Business Logic
Target: - contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol	

Description

Users can settle their long positions after short positions are settled. Once the user's position is settled, the user will not be able to settle again.

The problem here is that when the settlement token is on cross chain, malicious users can settle one empty short position for the victim. Then the victim will be marked as settled. When the victim wants to settle their long positions, their operations will be reverted.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L303-L344

```
function settleShort(
    address recipient,
    uint256 tokenAmount
) external payable nonReentrant whenNotPaused {
    uint256 totalTokenAmount = userShortBalance *
        ((_settlementTokenRatio * _settlementTokenTotalSupply) / 1 ether);
    _settledTokenAmount[recipient] += tokenAmount;
    if (_settledTokenAmount[recipient] >= totalTokenAmount) {
        _settled[recipient] = true;
    }
}
```

Recommendation

If the user's short balance is zero, the settlement for this short position should not be allowed.

Status

This issue has been resolved by the team with commit [de23323](#).

2. Settlement failure via manipulate options

Severity: High

Category: Business Logic

Target:

- contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol

Description

Users can settle their long positions after short positions are settled. Once the user's position is settled, the user will not be able to settle again.

Anyone can settle users' long positions via the function `settleLong`. The problem here is that users can manipulate options to cause settlement failure on the cross chain.

In LayerZero, users can use this option parameter to set the limited gas and send some native tokens to one specific address. Malicious users can set one quite low gas limit or choose to drop a tiny native asset to one address which denies receiving native ether.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L389-L477

```
function settleLong(
    bytes32 recipient,
    bytes memory options
) external payable nonReentrant whenNotPaused {
    settled[recipientAddress] = true;

    if (tokenAmount > 0) {
        if (_isCrossChain) {
            IBuildkeyLZRouter(authorizedLZRouter).transferTokens{
                value: msg.value
            }(
                _settlementTokenEid,
                _settlementTokenAddressBytes32,
                recipient,
                tokenAmount,
                options
            );
        }
    }
}
```

Recommendation

Add some input validation for input options.

Status

This issue has been resolved by the team with commit [de23323](#).

3. Temporary Dos via donating

Severity: Medium

Category: Business Logic

Target:

- contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol

Description

In AspectaTokenKeyPoolV2, users cannot open any short position if users have some remaining long positions.

The problem here is that anyone can buy keys for the victim. When users want to open one short position, malicious users can frontrun to buy one key for this victim. This will cause the victim to miss the opportunity to open the short position.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L184-L194

```
function buyByRouter(  
    uint256 amount,  
    address recipient  
) external payable override(AspectaTokenKeyPool) whenNotPaused {  
    _checkTradeStarted();  
    // make sure that the bought amount will not exceed the buy limit.  
    _checkKeyHoldLimit(recipient, amount);  
    amount = _clampBdcBuyAmount(amount);  
    _checkShortPosition(recipient);  
    _buy(recipient, amount);  
}
```

Recommendation

Add the related access control for function `buyByRouter`.

Status

This issue has been mitigated by the team.

4. Potential Asset Loss for AA Wallets via Cross-chain Settlement

Severity: Medium

Category: Business Logic

Target:

- contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol

Description

In the `settleLong` function, when `_isCrossChain` is enabled, the protocol automatically sends settlement tokens to the same address on the destination chain.

The implementation resolves the `recipientAddress` and passes it as the `recipient` parameter to the `authorizedLZRouter`.

While this works for EOAs (which have the same address across all chains), it is highly dangerous for Abstracted Accounts (AA).

AA wallet addresses are determined by their factory deployment parameters. If a user's wallet is not deployed with identical settings (factory address, salt, and bytecode) on the destination chain, the settlement tokens will be sent to an unowned or unclaimable address, leading to permanent loss of funds.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L389-L477

```
function settleLong(
    bytes32 recipient,
    bytes memory options
) external payable nonReentrant whenNotPaused {
    ...
    if (_isCrossChain) {
        // Cross-chain transfer
        IBuildkeyLZRouter(authorizedLZRouter).transferTokens(
            value: msg.value
        )(
            _settlementTokenEid,
            _settlementTokenAddressBytes32,
            recipient,
            tokenAmount,
            options
        );
    }
}
```

Recommendation

Modify the `settleLong` function to allow users to manually specify a `destinationRecipient` address instead of forcing the use of the source chain address.

Status

This issue has been resolved by the team with commit [de23323](#).

5. Unauthorized Trade Execution via tx.origin Phishing

Severity: Medium

Category: Business Logic

Target:

- contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol

Description

The `sellByRouter` function utilizes `tx.origin` to identify the seller in the internal `_sellInEth` call. This design choice creates a vulnerability where a malicious third-party contract can force a user to sell their tokens/keys without their explicit consent.

While the proceeds of the sale may still return to the user's wallet, the **timing, price, and decision to sell** are taken away from the user. A phishing contract can be designed to look like a completely different interaction (e.g., "Claim Reward"), but secretly trigger the `sellByRouter` function in the background.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L214-L225

```
function sellByRouter(  
    uint256 amount,  
    uint256 minPrice  
) external override(AspectaTokenKeyPool) whenNotPaused {  
    if (tgeTime > 0 && block.timestamp >= tgeTime + _settlementPeriod) {  
        revert TGEtimeReached();  
    }  
    _sellInEth(tx.origin, amount, minPrice);  
}
```

Recommendation

Replace `tx.origin` with `msg.sender`. If the function is intended to be called by a dedicated Router contract, implement a mapping of "Authorized Routers" or use the standard `allowance` (Approve/TransferFrom) pattern.

Status

This issue has been mitigated by the team.

6. Pool Insolvency due to Bonding Curve Price Hike via `setPiecewis`

Severity: Medium

Category: Business Logic

Target:

- `contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol`

Description

The protocol's solvency depends on the total ETH in the pool being sufficient to cover all potential settlements. A critical logic flaw exists when the ``owner`` (via ``setPiecewis``) increases the bonding curve price after a significant amount of Long positions have already been opened.

The Failure Scenario:

1. **Low Price Phase:** Many users open Long positions while the curve is flat (low price). The pool collects a small amount of ETH relative to the number of keys issued.
2. **Price Hike:** The ``owner`` calls ``setPiecewis`` to switch to a much steeper curve.
3. **Short Entry:** A user calls ``openShort``. The ``_openShort`` function calculates ``totalPrice`` based on the **new, higher price**.
4. **Collateral Locking:** The contract grants the short seller a credit of ``_shortDeposits[buyer] += 2 * totalPrice``.

Because ``totalPrice`` is now much higher than the ETH actually paid by the earlier Long holders for those same keys, the ``_shortDeposits`` liability recorded by the contract exceeds the actual ETH balance available in the pool.

`contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L36-L71`

```
function _openShort(
    address buyer,
    uint256 amount,
    uint256 minPrice
) internal nonReentrant {
    totalPrice = _calculateBdcSellPrice(amount);
    protocolFee = (totalPrice * protocolFeePercentage) / 1 ether;
    totalPriceWithFee = totalPrice + protocolFee;
    if (msg.value < totalPriceWithFee) {
        revert InsufficientFunds(totalPriceWithFee, msg.value);
    }
    _shortDeposits[buyer] += 2 * totalPrice;
}
```

Recommendation

Prevent any changes to the piecewise parameters if ``totalSupply()`` or ``_totalShortSupply`` is greater than zero.

Status

This issue has been mitigated by the team.

7. Centralization risk

Severity: Medium

Category: Centralization

Target:

- contracts/AspectaTokenKeyPool/AspectaTokenKeyPool.sol

Description

In the AspectaTokenKeyPool, BuildkeyLZRouter, BuildkeyTokenDispatcher contracts, there exists some privileged roles, `protocolFeeDestination`, `owner`. The privileged roles have the authority to execute some key functions such as `setPiecewisess`, `withdrawTokens`, `upgradeTo` etc.

If these roles' private keys are compromised, an attacker could trigger these functions to manipulate the price, steal the assets.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPool.sol:L370-L387

```
function setPiecewisess(  
    uint256[] calldata initialPrices,  
    uint256[] calldata boundaries,  
    uint256 triggerKeyAmount  
) external {  
    if (msg.sender != protocolFeeDestination) {  
        revert InvalidCaller();  
    }  
    if (_bdcTotalSupply < triggerKeyAmount) {  
        setPiecewisess(initialPrices, boundaries);  
    } else {  
        pendingBDCUpdate = PendingBDCUpdate({  
            initialPrices: initialPrices,  
            boundaries: boundaries,  
            triggerKeyAmount: triggerKeyAmount  
        });  
    }  
}
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been mitigated by the team.

8. Missing whenNotPaused Modifier in openShort and closeShort functions

Severity: Low

Category: Business Logic

Target:

- contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol

Description

The protocol implements a global `pause` functionality designed to halt activity during emergencies. While most critical state-changing functions in the `AspectaTokenKeyPool` utilize the `whenNotPaused` modifier, the `openShort` and `closeShort` functions lack this restriction.

This allows users to continue opening and closing short positions even when the contract is officially paused by the protocol owner or risk manager.

contracts/AspectaTokenKeyPoolV2/AspectaTokenKeyPoolV2.sol:L229-L254

```
function openShort(uint256 amount, uint256 minPrice) external payable {
    _checkTradeStarted();
    _checkPendingFairLaunchClaim(msg.sender);
    uint256 balanceAfter = _shortBalances[msg.sender] + amount;
    ...
    _openShort(msg.sender, amount, minPrice);
}
function closeShort(
    uint256 amount,
    uint256 maxPrice
) external returns (uint256 refund, uint256 remainingDeposit) {
    _checkTradeStarted();
    (refund, remainingDeposit) = _closeShort(msg.sender, amount, maxPrice);
}
```

Recommendation

Add the `whenNotPaused` modifier to both `openShort` and `closeShort` functions to ensure the pause mechanism covers all trading activities.

Status

This issue has been resolved by the team with commit [de23323](#).

2.3 Informational Findings

9. Redundant code

Severity: Informational

Category: Redundancy

Target:

- omni-hub/contracts/BuildkeyTokenDispatcher/BuildkeyDispatcher.sol

Description

The contract contains an import statement for ``forge-std/console.sol``. This library is part of the Foundry development framework and is used to print debug information to the terminal during local testing.

While the use of ``console.log`` typically does not introduce direct functional vulnerabilities, leaving the import and potential log statements in the codebase is considered a poor practice for production-grade contracts.

omni-hub/contracts/BuildkeyTokenDispatcher/BuildkeyDispatcher.sol:L19-L23

```
import {console} from "forge-std/console.sol";

contract BuildkeyTokenDispatcher is BuildkeyTokenDispatcherStorage {
    ...
}
```

Recommendation

Remove unnecessary code.

Status

This issue has been resolved by the team with commit [de23323](#).

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit [5fc9500](#):

File	SHA-1 hash
AspectaTokenKeyPoolV2Storage.sol	fb888323d7aef65ce84067d21b42243640221a58
IAspectaTokenKeyPoolV2.sol	ca7fe80b5a9f25161962292c1d1e41add0213750
AspectaTokenKeyPoolV2.sol	6c3cd5d9d1392e5239e4a11ad192e4215490dd0c
BuildkeyTokenDispatcherStorage.sol	4c1f962c6a7cc81028604f0210deac1478e937ac
IBuildkeyTokenDispatcher.sol	82e3b0b72a1d49a3a0ce3ae7621f8de8feb172b8
BuildkeyTokenDispatcher.sol	71e2ac09e82c8701e383825689d5fe7dbb195859
BuildkeyLZRouterStorage.sol	c256ad23466b758f42821ebe0ea2a4f68465f48c
IBuildkeyLZRouter.sol	325902640bc8c3388561416d602fcad8c25f324e
BuildkeyLZRouter.sol	64e116778d22b478130b9f67320302ba5cb862b9